YAML

CHEAT SHEETS

A Complete Reference for Modern YAML

Syntax · Scalars · Sequences · Mappings · Multiline Strings · Anchors & Aliases · Schema & Tags · Real-World Patterns · Tooling · Type Gotchas · Helm & Kustomize · yq · yamllint & Schemas

14 REFERENCE SHEETS

CONTENTS

CORE LANGUAGE

01 Basics & Syntax Rules
02 Scalars & Data Types
03 Collections — Sequences & Mappings
04 String Styles & Multiline
05 Anchors, Aliases & Merges

ADVANCED & APPLIED

06 Schema, Tags & Directives
07 Real-World YAML Patterns
08 Tooling, Validation & Language Integration

GOTCHAS & TOOLING

09 Type Gotchas & Quoting
10 Templating & Config Layering
11 yq — Query & Edit YAML
12 Editor Schemas & CI Linting

QUICK REFERENCE

13 Quick Reference — Syntax & Types
14 Quick Reference — Patterns & Tooling

Table of Contents

14 reference sheets · syntax · examples · notes

Sheet 1
Basics & Syntax Rules
syntax · indentation · document markers · comments · whitespace

Sheet 2
Scalars & Data Types
string · integer · float · boolean · null · timestamp · !! tags

Sheet 3
Collections — Sequences & Mappings
sequences · mappings · nesting · block style · flow style

Sheet 4
String Styles & Multiline
plain · quoted · literal block | · folded block > · chomping · escape sequences

Sheet 5
Anchors, Aliases & Merges
& anchor · * alias · << merge key · DRY config · override · multiple merges

Sheet 6
Schema, Tags & Directives
!! tags · %YAML · %TAG · core schema · custom tags · multi-document · Norway problem

Sheet 7
Real-World YAML Patterns
GitHub Actions · Docker Compose · Kubernetes · Ansible · OpenAPI · env config

Sheet 8
Tooling, Validation & Language Integration
yamllint · PyYAML · js-yaml · SnakeYAML · YAML↔JSON · validation · editors

Sheet 9
Type Gotchas & Quoting
Norway problem · yes / no / on / off · octal · sexagesimal · version strings · when to quote

Sheet 10
Templating & Config Layering
${VAR:-default} · .env · compose override · Helm {{ .Values }} · -f values.yaml · Kustomize overlays

Sheet 11
yq — Query & Edit YAML
yq eval · select() · = · |= · del() · -i · -o=json · ea · * merge · strenv()

Sheet 12
Editor Schemas & CI Linting
yaml-language-server: $schema · SchemaStore · yaml.schemas · .yamllint · pre-commit · check-jsonschema

Sheet 13
Quick Reference — Syntax & Types
structure · scalars · collections · quoting · block scalars · anchors & merges · tags · type gotchas

Sheet 14
Quick Reference — Patterns & Tooling
Docker Compose · GitHub Actions · Kubernetes · yq · yamllint · parsers by language · schemas · YAML vs JSON

Basics & Syntax Rules

WHAT IS YAML?

YAML (YAML Ain't Markup Language) is a human-readable data serialization format commonly used for **configuration files**, CI/CD pipelines, Infrastructure-as-Code, and data exchange between systems. YAML files use the extensions `.yml` or `.yaml` and are parsed by a YAML processor into native data structures (maps, lists, scalars). Unlike JSON, YAML is whitespace-sensitive — indentation defines structure, and **tabs are never allowed** for indentation.

CORE RULES

- Indentation uses `spaces only` — tabs cause parse errors
- Consistent indent depth required within a block (2 or 4 spaces typical)
- `#` starts a comment — extends to end of line
- Strings do not require quotes unless they contain special characters
- `---` marks the start of a document (optional for single-doc files)
- `...` marks the end of a document (optional)
- YAML is a superset of JSON — all valid JSON is valid YAML
- Keys and values are separated by `: ` (colon + space)

COMMON USE CASES

TOOL / PLATFORM	YAML ROLE
GitHub Actions	Workflow definition (.github/workflows/*.yml)
Docker Compose	Service, network, and volume config
Kubernetes	Resource manifests (Pod, Deployment, Service)
Ansible	Playbooks, inventory, variable files
OpenAPI / Swagger	API schema and endpoint definitions
Helm	Chart values and template overrides
Spring Boot	application.yml configuration
ESLint / Prettier	.eslintrc.yml, .prettierrc.yml

DOCUMENT MARKERS & SPECIAL SYNTAX

SYNTAX	MEANING
`---`	Start of document / separator between docs
`...`	Explicit end of document
`#`	Comment (inline or full line)
`:`	Key-value separator (colon + space required)
`-`	Sequence item (dash + space)
`|`	Literal block scalar (preserves newlines)
`>`	Folded block scalar (folds newlines)
`& / *`	Anchor definition / alias reference
`!!`	Explicit type tag (!!str, !!int, etc.)
`{ }`	Flow mapping (inline map)
`[ ]`	Flow sequence (inline list)

ANATOMY OF A YAML FILE

```yaml
%YAML 1.2                    # optional version directive
---                          # document start marker
# Application configuration
app:
  name: my-service           # string value (unquoted)
  version: 2.1.0             # string (looks like semver)
  port: 8080                 # integer
  debug: false               # boolean

server:
  host: "localhost"          # quoted string
  timeout: 30                # integer (seconds)

features:                    # mapping with list value
  - authentication
  - rate-limiting
  - logging

database:
  url: postgres://db:5432    # colon in value — no quotes needed here
  pool_size: 10

# Inline (flow) style — equivalent to block above
tags: {env: prod, region: us-east}
ports: [80, 443, 8080]
...                          # optional document end marker
```

NOTES

- `%YAML 1.2` — optional directive; most parsers default to 1.1 or 1.2
- `---` — required when using directives or multiple documents; otherwise optional
- `app:` — a mapping key; child keys indented beneath it form a nested map
- `false` — unquoted booleans are auto-detected; quote to force string: `"false"`
- `features:` — a mapping whose value is a sequence (list of strings)
- `{ }` / `[ ]` — flow style keeps items on one line; valid anywhere block style is
- Colons inside values don't need quoting unless they appear at the start or are followed by a space

COMMON MISTAKES

- Using `Tab` for indentation — YAML forbids tabs entirely; always use spaces
- Forgetting the space after `:` — `key:value` is a string, not a mapping
- Inconsistent indentation depth within a block causes parse errors
- Writing `: #comment` directly — always leave a space: `value # comment`

TIPS

- Use 2-space indentation — it's the community standard and most tools default to it
- Configure your editor to show whitespace characters to catch tab vs space issues early
- Run `yamllint` on files before committing — catches indentation and syntax errors instantly
- Use `---` at the top of every file as a habit — it makes multi-doc scenarios trivial to add later

SUMMARY

- `spaces only` — tabs are illegal for indentation
- `: ` — colon + space separates every key from its value
- `#` — comments extend to end of line
- `---` / `...` — document start and end markers
- Block style uses indentation; flow style uses `{}` and `[]`
- YAML is a superset of JSON

Scalars & Data Types

SCALAR TYPES

A **scalar** is any single-value node in YAML — a string, number, boolean, or null. YAML performs **implicit type detection**: the parser infers the type from the value's form unless you override it with an explicit tag. This convenience is also a source of bugs — values like `yes`, `on`, `1.0`, and `2024-01-01` are automatically coerced to boolean, float, and timestamp respectively. Use **explicit tags** (`!!str`, `!!int`) or quotes to force a specific type.

TYPE DETECTION RULES

- Quoted values (`'…'` or `"…"`) are always strings
- `true` , `false` → boolean (YAML 1.2); also `yes` / `no` , `on` / `off` in YAML 1.1
- `null` , `~` , or empty value → null type
- Bare integers and floats are numeric unless quoted
- `YYYY-MM-DD` format → timestamp (date)
- `0x1F` → hex integer; `0o17` → octal
- `.inf` , `-.inf` , `.nan` → special float values

NOTES

→ `''` — single-quote escape in single-quoted strings (double the quote)

→ `\n \t \uXXXX` — escape sequences only work inside double-quoted strings

→ `!!str 1984` — explicit tag overrides auto-detection; value becomes the string "1984"

→ `0xFF` / `0o17` — hex and octal integer literals; not all parsers support these

→ `.inf` / `.nan` — case-insensitive; also `.Inf` , `.NaN` , `.INF`

→ `true` / `false` only — YAML 1.2 dropped yes/no/on/off as booleans; YAML 1.1 parsers still accept them

→ Empty value after `:` is null — not an empty string; use `""` for an empty string

SCALAR EXAMPLES & TYPE COERCION

```yaml
# ── Strings ──────────────────────────────────────
plain: hello world              # string (no quotes needed)
single: 'it''s alive'           # single quotes: '' escapes a quote
double: "tab:\there\nnewline"   # double quotes: escape sequences active
forced_str: !!str 1984          # force "1984" not integer 1984

# ── Numbers ──────────────────────────────────────
count: 100
hex_val: 0xFF                   # 255
octal_val: 0o17                 # 15
pi: 3.14159
scientific: 6.022e23
infinity: .inf
not_a_number: .nan

# ── Booleans (YAML 1.2 — only true/false) ────────
enabled: true
disabled: false
as_string: "true"               # quoted → stays a string

# ── Null ─────────────────────────────────────────
nothing: null
tilde: ~
implicit:                       # empty value → null

# ── Timestamps ───────────────────────────────────
date: 2024-06-01
datetime: 2024-06-01T14:30:00Z
datetime_tz: 2024-06-01T14:30:00+05:30
date_str: !!str 2024-06-01      # force string, not date
```

SCALAR TYPES & EXAMPLES

TYPE	EXAMPLE YAML	PARSED AS
String	hello world	"hello world"
String	'can''t stop'	"can't stop"
String	"line\nbreak"	line + newline + break
Integer	42	42 (int)
Integer	0xFF	255 (hex)
Integer	0o77	63 (octal)
Float	3.14	3.14 (float)
Float	1.5e3	1500.0
Float	.inf	Infinity
Float	.nan	NaN
Boolean	true / false	true / false
Null	null / ~ / (empty)	null
Timestamp	2024-06-01	date object
Timestamp	2024-06-01T12:00:00Z	datetime (UTC)

EXPLICIT TYPE TAGS (!!)

TAG	FORCES TYPE	EXAMPLE
`!!str`	String	!!str 42 → "42"
`!!int`	Integer	!!int "10" → 10
`!!float`	Float	!!float 1 → 1.0
`!!bool`	Boolean	!!bool "yes" → true
`!!null`	Null	!!null "" → null
`!!seq`	Sequence	forces list type
`!!map`	Mapping	forces map type
`!!binary`	Base64 blob	binary data
`!!timestamp`	Date/time	datetime object
`!!python/...`	Python type	PyYAML extension

COMMON MISTAKES

- `yes` / `no` become booleans in YAML 1.1 parsers — quote them if you mean the string "yes"
- Version strings like `1.10` parse as float `1.1` — always quote version numbers
- Empty value is `null` , not `""` — use explicit `""` when an empty string is required
- Timestamps are auto-parsed — quote ISO dates when you need strings (e.g. filenames)

TIPS

- When in doubt, quote it — quoted scalars are always strings regardless of content
- Use `!!str` tags at the key level for self-documenting intent when type is non-obvious
- Know your parser's YAML version — PyYAML defaults to 1.1; `ruamel.yaml` and most modern parsers support 1.2
- Single quotes are safer than double quotes when values contain backslashes (no escape processing)

SUMMARY

- `implicit typing` — parser infers type from value shape
- `!!str` / `!!int` — explicit tags override inference
- `true` / `false` — only booleans in YAML 1.2
- `null` / `~` / empty — all represent null
- `YYYY-MM-DD` — auto-parsed as timestamp; quote to prevent

Sheet 03

Collections — Sequences & Mappings

sequences · mappings · nesting · block style · flow style

THE TWO COLLECTION TYPES

YAML has two collection types: **sequences** (ordered lists) and **mappings** (key-value dictionaries). Both can be written in **block style** (multi-line, indented) or **flow style** (inline, using `[]` and `{}`). Collections nest freely — a mapping value can be a sequence, a sequence item can be a mapping, and any depth of nesting is valid. The indentation level of a child must be *greater than* its parent's key.

COLLECTION RULES

- Sequence items start with `- ` (dash + space)
- Mapping entries use `key: value` (colon + space)
- Flow sequences: `[a, b, c]` — comma-separated in brackets
- Flow mappings: `{key: val, key2: val2}`
- Keys must be unique within the same mapping level
- Keys can be any scalar (string, int, bool) — strings most common
- Complex keys (multi-line) use the `? ` indicator
- Empty sequence: `[]` · Empty mapping: `{}`

NOTES

→ `- item` — the dash is part of the indentation; the value starts after the space

→ `server:` then indented children — the mapping's value is itself a mapping

→ `users:` + `- name:` — sequence of mappings; each `-` begins a new object

→ The `-` indicator counts as 2 spaces of indentation for its children

→ `environments: production: servers:` — three levels of nested mappings

→ `[[1, 0], [0, 1]]` — flow sequences can nest; useful for matrices or coordinate pairs

→ Block and flow styles can be mixed freely — a block mapping can have a flow sequence as a value

NESTING COLLECTIONS

```yaml
# ── Simple sequence ──────────────────────────
fruits:
  - apple
  - banana
  - cherry

# ── Flow sequence (equivalent) ───────────────
fruits: [apple, banana, cherry]

# ── Mapping with nested mapping ──────────────
server:
  host: localhost
  port: 8080
  tls:
    enabled: true
    cert: /etc/ssl/cert.pem

# ── Sequence of mappings (list of objects) ───
users:
  - name: Alice
    role: admin
    active: true
  - name: Bob
    role: viewer
    active: false

# ── Mixed nesting: map → list → map ──────────
environments:
  production:
    servers:
      - host: prod-1.example.com
        weight: 100
      - host: prod-2.example.com
        weight: 100
  staging:
    servers:
      - host: stage.example.com
        weight: 100

# ── Flow style for compact data ──────────────
matrix: [[1, 0], [0, 1]]
rgb: {r: 255, g: 128, b: 0}
```

SEQUENCE PATTERNS

PATTERN	YAML
Block list	`– item1` / `– item2`
Flow list	`[item1, item2, item3]`
Empty list	`key: []`
Nested lists	Indent – items under a –
List of maps	`– name: Alice` / `age: 30`
Inline list of maps	`– {name: Alice, age: 30}`
Single-item list	`– only_item`

MAPPING PATTERNS

PATTERN	YAML
Simple map	`key: value`
Flow map	`{a: 1, b: 2}`
Empty map	`key: {}`
Nested map	Child keys indented under parent key
Map with list value	`ports:` then `– 80` / `– 443`
Integer key	`404: Not Found`
Complex key	`? [1, 2]` / `: value`

COMMON MISTAKES

- Not indenting map children — siblings at the same level as the parent key break the structure
- Forgetting the space after `-` in sequences — `-item` is a string, not a list item
- Duplicate keys in the same mapping — most parsers silently use the last value; behavior is undefined
- Mixing indentation depths within the same block — 2-space parent cannot have 3-space children

TIPS

- Use block style for config files; reserve flow style for short, simple values that read naturally inline
- Sequence-of-mappings is the most common YAML pattern — master `- key: value` nesting first
- Paste YAML into an online validator (yaml-online-parser.appspot.com) to visualize the parsed tree

SUMMARY

- `- item` — block sequence item
- `[a, b]` — flow sequence
- `key: value` — mapping entry
- `{k: v}` — flow mapping
- Collections nest to any depth; indentation defines the parent-child relationship

String Styles & Multiline

plain · single-quoted · double-quoted · literal | · folded > · chomping

FIVE WAYS TO WRITE A STRING

YAML provides five string styles. **Plain** scalars need no quotes and are the most readable. **Single-quoted** strings treat all characters literally (no escape sequences) except `''` for a literal quote. **Double-quoted** strings support backslash escapes (`\n`, `\t`, `\uXXXX`). **Literal block** (`|`) preserves newlines exactly — ideal for scripts or file contents. **Folded block** (`>`) converts single newlines to spaces and double newlines to paragraph breaks — ideal for long prose.

ESCAPE SEQUENCES (DOUBLE-QUOTED ONLY)

- `\n` — newline
- `\t` — tab
- `\\` — literal backslash
- `\"` — literal double quote
- `\r` — carriage return
- `\0` — null character
- `\uXXXX` — Unicode code point (4 hex digits)
- `\UXXXXXXXX` — Unicode code point (8 hex digits)
- `\xXX` — 8-bit Unicode (2 hex digits)

NOTES

- → `'C:\Users\...'` — single quotes are the safest choice for Windows paths; no escape processing
- → `''` — the only escape in single-quoted strings; doubles a single quote character
- → `|` — literal block; content indented below the key, indentation is stripped from output
- → `|-` — strip modifier removes all trailing newlines; useful when embedding in templates
- → `>` — folded block; single line breaks become spaces; blank line creates `\n\n`
- → `>-` — folded + strip; most common for long single-line config values
- → Block scalar indentation is detected automatically from the first content line; or set explicitly with `|2`

ALL STRING STYLES COMPARED

```yaml
# ── Plain scalar ──────────────────────────────────────
greeting: Hello World           # → "Hello World"
version: "1.0"                  # quoted to prevent float parse

# ── Single-quoted ─────────────────────────────────────
path: 'C:\Users\admin\file.txt' # backslash literal, no escaping
quote: 'she said ''hello'''     # '' inside single quotes = literal '

# ── Double-quoted ─────────────────────────────────────
newlines: "first\nsecond\nthird"  # → first⏎second⏎third
unicode: "\u00A9 2024"            # → © 2024
tab: "col1\tcol2"                 # → col1⇥col2

# ── Literal block (|) — newlines preserved ────────────
script: |
  #!/bin/bash
  echo "Starting..."
  cd /app && npm start
  # Each line kept exactly, including the newline at end

# ── Literal strip (|-) — trailing newlines removed ────
inline_script: |-
  echo "no trailing newline"
  exit 0

# ── Folded block (>) — single newlines become spaces ──
description: >
  This is a long description
  that spans multiple lines but
  will be joined into one paragraph.

  A blank line creates a new paragraph.

# ── Folded strip (>-) — fold + no trailing newline ────
summary: >-
  One long line with
  no trailing newline at end
```

STRING STYLE COMPARISON

STYLE	INDICATOR	ESCAPE SEQS	BEST FOR
Plain	none	✗	Simple words, most values
Single-quoted	'…'	✗	Strings with special chars, no backslash
Double-quoted	"…"	✔	Strings needing \n, \t, unicode
Literal block	|	✗	Scripts, file content, preserve newlines
Folded block	>	✗	Long prose, word-wrapped text

BLOCK CHOMPING INDICATORS

INDICATOR	TRAILING NEWLINES	EXAMPLE
| or >	One newline kept (clip, default)	| >
|- or >-	All trailing newlines removed (strip)	|– >–
|+ or >+	All trailing newlines kept (keep)	|+ >+
|2 or |4	Explicit indentation (2 or 4 spaces)	|2 |4
|2-	Indent indicator + strip combined	|2–

COMMON MISTAKES

- Using `|` when you want a single-line result — use `>-` to fold lines and strip the trailing newline
- Expecting `\n` to work in single-quoted strings — escapes only work in double-quoted strings
- Inconsistent indentation inside block scalars — the first content line sets the indentation baseline
- Not accounting for the trailing newline in `|` blocks when passing to shell commands

TIPS

- Use `|-` for embedded scripts and commands — it removes the trailing newline that would otherwise be passed to the shell
- Use `>-` for long descriptions in CI/CD configs — keeps the file readable while producing a single string
- Single-quoted strings are ideal for regex patterns and file paths — no backslash surprises

SUMMARY

- `plain` — no quotes; simplest; auto-typed
- `'…'` — single-quoted; literal backslashes; `''` for quote
- `"…"` — double-quoted; full escape sequences
- `|` — literal block; preserves newlines
- `>` — folded block; single breaks → spaces
- `-` modifier strips trailing newlines; `+` keeps them all

Anchors, Aliases & Merges

& anchor · * alias · << merge key · DRY config · override · multiple merges

REUSE WITHOUT REPETITION

YAML's **anchor** (&name) and **alias** (*name) mechanism lets you define a value once and reference it multiple times, keeping configuration DRY. An anchor marks a node; an alias inserts a copy of that node elsewhere in the document. The **merge key** (<<) extends this by merging a mapping's contents into another mapping, with the ability to override specific keys after the merge. Anchors and aliases are resolved at parse time — the consumer sees fully expanded data.

ANCHOR & ALIAS RULES

- `&name` — defines an anchor; name can contain letters, digits, hyphens
- `*name` — inserts a copy (alias) of the anchored node
- Anchors must appear *before* their aliases in the document
- Aliases produce a deep copy — modifying one does not affect the other (in most parsers)
- `<<: *anchor` — merge key; merges anchored mapping into current mapping
- Keys defined after a merge override the merged values
- Multiple merges: `<<: [*a, *b]` — first list item has highest priority
- Anchors work on scalars, sequences, and mappings

NOTES

- → `&defaults` — anchor name follows immediately; the mapping below it is the anchored node
- → `<<: *defaults` — merge key; all keys from the anchor are inserted into the current map
- → `timeout: 60` after merge — explicit keys after `<<` override merged values
- → `[*base, *monitoring]` — multiple merge; first anchor's keys take priority over later anchors on conflict
- → Aliases in string values (`"https://*domain"`) do NOT expand — `*name` only works at the node level
- → `*ports` as a value inserts the full sequence — each service gets its own copy of the list
- → The merge key `<<` is only valid in mappings — it has no effect in sequences

ANCHORS, ALIASES & MERGE KEYS

```yaml
# ── Define a shared anchor ──────────────────────────
defaults: &defaults
  retries: 3
  timeout: 30
  log_level: info

# ── Simple alias — exact copy ───────────────────────
worker_a:
  <<: *defaults          # merges retries, timeout, log_level
  queue: high-priority   # adds a new key

worker_b:
  <<: *defaults
  queue: low-priority
  timeout: 60            # overrides the merged timeout: 30

# ── Multiple merges ─────────────────────────────────
base: &base
  image: ubuntu:22.04
  restart: unless-stopped

monitoring: &monitoring
  metrics: true
  health_check: /healthz

app:
  <<: [*base, *monitoring]   # both merged; base keys win on conflict
  name: my-app
  port: 8080

# ── Scalar anchor ───────────────────────────────────
domain: &domain example.com
api_url: "https://*domain/api"  # NOTE: aliases in strings don't expand
                                # use anchors only at node level

# ── Sequence alias ──────────────────────────────────
common_ports: &ports [80, 443, 8080]
service_a:
  ports: *ports
service_b:
  ports: *ports
```

ANCHOR / ALIAS / MERGE SYNTAX REFERENCE

SYNTAX	ROLE	EXAMPLE
`&name`	Define anchor on a node	`defaults: &defaults`
`*name`	Insert alias (copy of anchor)	`prod: *defaults`
`<<: *name`	Merge anchor mapping into current mapping	`<<: *defaults`
`<<: [*a, *b]`	Merge multiple anchors (first wins on conflict)	`<<: [*base, *extra]`
`key: *name`	Alias as a value in a mapping	`config: *shared`
`- *name`	Alias as a sequence item	`- *server_defaults`

COMMON MISTAKES

- Placing an alias before its anchor — anchors must be defined earlier in the document
- Expecting `"https://*domain"` string interpolation — aliases don't expand inside quoted strings
- Overriding a key before the merge — put `<<` first, then override keys below it
- Multiple `<<` keys in the same map — only one merge key is valid; use the list form `[*a, *b]`

TIPS

- Use a top-level `x-` prefixed key (e.g. `x-defaults:`) for anchors you don't want parsed as real data — Docker Compose ignores `x-*` keys
- Define all anchors near the top of the file so they're easy to find and maintain
- Merge keys work in Kubernetes Helm values — useful for environment-specific overrides

SUMMARY

- `&name` — define a reusable anchor
- `*name` — insert alias (copy) of anchor
- `<<: *name` — merge anchor mapping into current map
- Keys after merge override merged values
- `[*a, *b]` — merge multiple anchors; first wins on conflict

Schema, Tags & Directives

SCHEMAS & TYPE RESOLUTION

A YAML **schema** defines how untagged scalars are resolved to types. The **Failsafe schema** treats everything as a string. The **JSON schema** recognizes true/false/null/numbers exactly as JSON does. The **Core schema** (YAML 1.2 default) extends JSON with additional true/false/null forms and number formats. The **YAML 1.1 schema** (still used by many parsers) additionally recognizes `yes/no/on/off` as booleans — the source of many cross-version bugs. Explicit **tags** (`!!str`, `!mytype`) override schema resolution entirely.

SCHEMA & DIRECTIVE RULES

- `%YAML 1.2` directive declares the YAML version; must appear before `---`
- `%TAG ! tag:yaml.org,2002:` — shorthand for built-in tag prefix
- `!!` is shorthand for `tag:yaml.org,2002:` — so `!!str` = full URI tag
- Custom tags use single `!`: `!mytag value`
- Tags apply to any node: scalars, sequences, and mappings
- Parsers may not support all tags — check your library's docs
- Multi-document: multiple `---` separated docs in one file

NOTES

- → `%YAML 1.2` — directive tells the parser which schema to use; affects boolean/null/int resolution
- → `%TAG !` — defines a shorthand prefix for custom tag URIs (! is the default handle)
- → `!!int "8080"` — tag on a quoted string; forces it to integer 8080 despite the quotes
- → `!db-config` — application-defined tag; Python's PyYAML can map this to a class constructor
- → `!!binary |` — base64-encoded binary data in a literal block; parsers decode to bytes
- → `---` between documents — each block is a separate parsed document; use `load_all()` in most parsers
- → The Norway problem: country code `NO` was parsed as boolean false in YAML 1.1 — always quote country codes

DIRECTIVES, TAGS & MULTI-DOCUMENT

```yaml
%YAML 1.2
%TAG ! tag:example.com,2024:
---
# ── Explicit built-in tags ──────────────────────
port:     !!int "8080"          # force integer
version:  !!str 2.0             # force string "2.0"
enabled:  !!bool "yes"          # force boolean true
data:     !!null ""             # force null

# ── Custom application tag ──────────────────────
connection: !db-config
  host: localhost
  driver: postgres

# ── Binary data tag ─────────────────────────────
thumbnail: !!binary |
  R0lGODlhDAAMAIQAAP//9/X
  17unp5WZmZgAAAOfn515eXv
  Pz7Y6OjuDg4J+fn5OTk6enp
  56SkpGRkZCQkAAAACH5BAEAA
  BQALAAAAAAMAAwAAAVWoCAA

# ── Multi-document file ─────────────────────────
---
# Document 1: development config
env: development
debug: true
db_url: sqlite:///dev.db

---
# Document 2: production config
env: production
debug: false
db_url: postgres://prod-db:5432/app
```

BUILT-IN TAGS (!! SHORTHAND)

TAG	FULL URI	RESULT TYPE
`!!str`	tag:yaml.org,2002:str	String
`!!int`	tag:yaml.org,2002:int	Integer
`!!float`	tag:yaml.org,2002:float	Float
`!!bool`	tag:yaml.org,2002:bool	Boolean
`!!null`	tag:yaml.org,2002:null	Null
`!!seq`	tag:yaml.org,2002:seq	Sequence
`!!map`	tag:yaml.org,2002:map	Mapping
`!!binary`	tag:yaml.org,2002:binary	Base64 binary data
`!!timestamp`	tag:yaml.org,2002:timestamp	Datetime object
`!!omap`	tag:yaml.org,2002:omap	Ordered map

SCHEMA GOTCHAS (THE NORWAY PROBLEM & OTHERS)

VALUE	YAML 1.1	YAML 1.2
`NO`	boolean false	string "NO"
`yes`	boolean true	string "yes"
`on`	boolean true	string "on"
`off`	boolean false	string "off"
`1.0`	float 1.0	float 1.0
`1_000`	integer 1000	string "1_000"
`0777`	octal 511	integer 777
`1:30`	integer 90 (sexagesimal)	string "1:30"

COMMON MISTAKES

- Using `NO`, `yes`, `on`, `off` as strings in YAML 1.1 parsers — they silently become booleans
- Assuming `%YAML 1.2` directive changes behavior in all parsers — PyYAML ignores it and uses 1.1 rules
- Forgetting `load_all()` for multi-document files — `load()` only returns the first document
- Using `0777` for octal — YAML 1.2 treats this as integer 777; use `0o777` instead

TIPS

- Use `ruamel.yaml` instead of `PyYAML` when you need true YAML 1.2 compliance in Python
- Quote all ISO country codes, version numbers, and date-like strings to prevent type coercion surprises
- Multi-document YAML is useful for Kubernetes manifests — multiple resources in a single `kubectl apply -f` file

SUMMARY

- `%YAML 1.2` — version directive before `---`
- `!!` — shorthand for built-in YAML tag namespace
- `!mytag` — custom application-defined tag
- YAML 1.1 vs 1.2 — yes/no/on/off behavior differs significantly
- `---` separates multiple documents in one file

Real-World YAML Patterns

YAML IN THE WILD

YAML is the dominant configuration language for modern DevOps, cloud infrastructure, and API tooling. Each platform has its own schema and required top-level keys — but they all share the same YAML syntax rules. Understanding the **structural patterns** of each platform helps you read unfamiliar configs quickly and write new ones without referring to documentation for every field. The code block shows minimal but complete examples of the most common platforms.

MINIMAL REAL-WORLD EXAMPLES

```yaml
# ── GitHub Actions workflow ──────────────────────
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm test

# ── Docker Compose service ───────────────────────
services:
  web:
    image: nginx:alpine
    ports: ["80:80"]
    volumes: [./html:/usr/share/nginx/html:ro]
    depends_on: [api]
  api:
    build: ./api
    environment:
      DATABASE_URL: postgres://db:5432/app

# ── Kubernetes Deployment (excerpt) ──────────────
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  labels: {app: my-app}
spec:
  replicas: 3
  selector:
    matchLabels: {app: my-app}
  template:
    spec:
      containers:
        - name: app
          image: my-app:latest
          ports: [{containerPort: 8080}]

# ── Ansible task ─────────────────────────────────
- name: Install nginx
  hosts: webservers
  become: true
  tasks:
    - name: Install package
      ansible.builtin.apt:
        name: nginx
        state: present
```

NOTES

- → `on: [push, pull_request]` — GitHub Actions: note `on` is also a YAML 1.1 boolean — quote it if your parser is strict: `"on":`
- → `uses: actions/checkout@v4` — references a reusable action from GitHub Marketplace
- → `depends_on` — Docker Compose: ensures `db` starts before `api`, but does not wait for readiness
- → `apiVersion: apps/v1` — Kubernetes: version determines which fields are available; always check the API docs
- → `matchLabels` — Kubernetes selector must exactly match the pod template's labels or pods won't be managed
- → `become: true` — Ansible: escalates to root (sudo); required for package installation
- → `ansible.builtin.apt` — fully-qualified module name; preferred in modern Ansible over bare `apt:`

PLATFORM KEY FIELDS

- **GitHub Actions:** `on`, `jobs`, `steps`, `uses`, `run`
- **Docker Compose:** `version`, `services`, `networks`, `volumes`
- **Kubernetes:** `apiVersion`, `kind`, `metadata`, `spec`
- **Ansible:** `hosts`, `become`, `tasks`, `vars`, `handlers`
- **OpenAPI 3:** `openapi`, `info`, `paths`, `components`
- **Helm values:** plain YAML; referenced in templates with `{{ .Values.key }}`

PLATFORM STRUCTURE SUMMARY

PLATFORM	ROOT KEYS	FILE
GitHub Actions	name · on · env · jobs	.githu
Docker Compose	version · services · networks · volumes	dock
Kubernetes Pod	apiVersion · kind · metadata · spec	any .y
Kubernetes Deployment	same + spec.template.spec.containers	deplo
Ansible Playbook	hosts · become · vars · tasks · handlers	playb
OpenAPI 3.x	openapi · info · servers · paths · components	open
Helm values	user-defined; referenced via {{ .Values.x }}	value
ESLint	env · extends · rules · plugins	.eslin

COMMON MISTAKES

- Using `on:` unquoted in GitHub Actions with YAML 1.1 parsers — some tools parse it as boolean `true`
- Kubernetes `matchLabels` not matching `template.metadata.labels` — deployment silently manages zero pods
- Docker Compose `depends_on` only controls start order, not readiness — use healthchecks for true readiness gating
- Ansible tasks list is a sequence — forgetting the leading `-` on each task causes a parse or runtime error

TIPS

- Use `kubectl apply --dry-run=client -f manifest.yaml` to validate Kubernetes YAML without deploying
- Use `docker compose config` to validate and print the fully-merged Compose file before running
- Use `ansible-playbook --syntax-check playbook.yml` to catch YAML and Ansible structural errors early

SUMMARY

- GitHub Actions: `on · jobs · steps · uses · run`
- Docker Compose: `services · image · ports · volumes`
- Kubernetes: `apiVersion · kind · metadata · spec`
- Ansible: `hosts · become · tasks` (sequence of maps)
- Each platform has a strict schema — validate before applying

Tooling, Validation & Language Integration

PARSING & VALIDATION

Every language has at least one YAML library, but they differ in **YAML version support**, **security defaults**, and **round-trip fidelity**. PyYAML is the most common Python library but defaults to YAML 1.1 and has unsafe loaders. `ruamel.yaml` provides true YAML 1.2 and preserves comments on round-trip. In JavaScript, `js-yaml` is the de facto standard. Always use the **safe** loader/parser variant when loading untrusted input — unsafe loaders can execute arbitrary code via custom tags.

SAFETY & BEST PRACTICES

- Never use `yaml.load()` in Python — always use `yaml.safe_load()`
- `yaml.safe_load()` only resolves standard YAML types; no custom Python objects
- In Node.js, `js-yaml` uses safe schema by default
- Run `yamllint` in CI to catch syntax and style issues before merge
- `yamllint -d relaxed file.yml` — relaxed profile allows more flexible formatting
- VS Code: install `YAML` extension by Red Hat for schema validation and autocompletion
- JSON Schema can validate YAML — many tools support `$schema` in YAML files

NOTES

- → `yaml.safe_load()` — only use `yaml.load()` with `Loader=yaml.SafeLoader`; bare `load()` can execute code
- → `yaml.safe_load_all()` — returns a generator of documents from a multi-document file
- → `default_flow_style=False` — forces block style output; `True` produces compact inline YAML
- → `ruamel.yaml` — preserves comments, quote styles, and key ordering on round-trip writes
- → `yml.load(f)` — ruamel's load is safe by default when using the `YAML()` constructor with no type argument
- → For JavaScript: `const yaml = require('js-yaml'); yaml.load(str)` — same pattern, safe by default
- → For Go: annotate structs with `yaml:"field_name"` tags then call `yaml.Unmarshal(data, &cfg)`

READING & WRITING YAML IN CODE

```python
# ── Python: PyYAML ──────────────────────────────────────
import yaml

# Read YAML file safely
with open("config.yml") as f:
    config = yaml.safe_load(f)      # returns dict
    # config["app"]["port"] → 8080

# Read YAML string
data = yaml.safe_load("name: Alice\nage: 30")
# → {"name": "Alice", "age": 30}

# Write YAML
output = yaml.dump({"key": "value", "num": 42})
# → "key: value\nnum: 42\n"

# Pretty-print with options
yaml.dump(data, default_flow_style=False, indent=2)

# Multiple documents
docs = list(yaml.safe_load_all(open("multi.yml")))


# ── Python: ruamel.yaml (YAML 1.2 + comment-preserving) ─
from ruamel.yaml import YAML
yml = YAML()
yml.preserve_quotes = True

with open("config.yml") as f:
    data = yml.load(f)              # preserves comments!

data["version"] = "2.0"
with open("config.yml", "w") as f:
    yml.dump(data, f)               # comments stay intact
```

VALIDATION & CLI TOOLS

TOOL	INSTALL	USAGE
`yamllint`	pip install yamllint	yamllint file.yml
`yq`	brew install yq	yq '.key' file.yml
`python -c`	built-in	python -c "import yaml,sys; yaml.safe_load(sys.stdin)"
`js-yaml CLI`	npm i -g js-yaml	js-yaml file.yml
`kubectl validate`	kubectl	kubectl apply --dry-run=client -f f.yaml
`cfn-lint`	pip install cfn-lint	cfn-lint template.yaml (CloudFormation)

LIBRARY QUICK REFERENCE

LANGUAGE	LIBRARY	READ	WRITE
Python	PyYAML	safe_load()	dump()
Python	ruamel.yaml	load()	dump()
JavaScript	js-yaml	load()	dump()
Go	gopkg.in/yaml.v3	Unmarshal()	Marshal()
Java	SnakeYAML	load()	dump()
Ruby	Psych (stdlib)	YAML.load()	to_yaml
Rust	serde_yaml	from_str()	to_string()
.NET	YamlDotNet	Deserializer	Serializer

COMMON MISTAKES

- Using bare `yaml.load()` in Python — always use `yaml.safe_load()` or pass `Loader=yaml.SafeLoader`
- Expecting `PyYAML` to honor `%YAML 1.2` — it doesn't; use `ruamel.yaml` for 1.2 compliance
- Using `yaml.load_all()` without iterating — it returns a lazy generator, not a list
- Round-tripping with PyYAML loses comments, custom tags, and formatting — use ruamel.yaml when preserving these matters

TIPS

- Use `yq` on the command line to query and transform YAML files — `yq '.services.web.image' docker-compose.yml`
- Add `yamllint` to your pre-commit hooks — catches indentation errors before they reach CI
- Install the VS Code Red Hat YAML extension and point it at JSON Schema files for inline validation and autocomplete

SUMMARY

- `yaml.safe_load()` — always use safe loader in Python
- `ruamel.yaml` — YAML 1.2 + round-trip comment preservation
- `js-yaml` — standard JavaScript YAML parser
- `yamllint` — lint YAML files in CI pipelines
- `yq` — jq-style CLI tool for querying YAML files

Type Gotchas & Quoting

Norway problem · yes / no / on / off · octal · sexagesimal · version strings · when to quote

PLAIN SCALARS GUESS THEIR TYPE

An unquoted (**plain**) scalar is typed by pattern matching: `true` becomes a boolean, 42 an integer, ~ null. The patterns differ between **YAML 1.1** (PyYAML, Ruby's Psych, many older tools) and **YAML 1.2** (js-yaml 4, the `yaml` npm package, ruamel.yaml), so the same file can load differently in two tools. Country codes, version numbers, port mappings and zip codes are the usual victims. The fix is almost always the same: **quote anything that must stay a string**.

QUICK RULES

- Quote values that look like booleans, numbers, dates or nulls but are really text
- Quote strings that start with `@`, backtick, `*`, `&`, `!`, `|`, `>`, `%`, `{`, `[` or `#`
- Quote strings containing `: ` or `#` — they start a mapping or a comment
- A colon *without* a following space is fine: `url: http://x.io:8080`
- Single quotes are literal (`''` escapes a quote); double quotes allow `\n`, `\t`, `\u00e9`

NOTES

- → **Norway problem** — `NO` is a 1.1 boolean, so a list of country codes turns Norway into `false`
- → `yes / no / on / off` — 1.1 booleans in any case (`Yes`, `OFF`); plain strings in 1.2
- → `010` — 1.1 reads a leading zero as octal; 1.2 needs the `0o` prefix
- → `22:22` — 1.1 reads colon-separated digits as base-60; Docker Compose docs tell you to quote ports
- → `3.10` — a float in every version; GitHub Actions `python-version: 3.10` installs 3.1
- → `on:` as a key — PyYAML loads the GitHub Actions trigger key as `True`; GitHub's parser doesn't
- → Empty value and `~` — both are null; use `""` for an empty string

SAME TEXT, DIFFERENT TYPES

```yaml
#                          YAML 1.1 (PyYAML)      YAML 1.2 core
country: NO                # false                "NO"
answer:  yes               # true                 "yes"
switch:  off               # false                "off"
octal:   010               # 8                    10
octal2:  0o10              # "0o10"               8
port:    22:22             # 1342  (base 60!)     "22:22"
version: 3.10              # 3.1                  3.1      ← both!
sci:     1e3               # "1e3"                1000.0
date:    2026-10-23        # datetime.date        "2026-10-23"
zip:     02134             # 1116 (octal)         2134
empty:                     # None                 null
tilde:   ~                 # None                 null

# Safe in every parser:
country: "NO"
version: "3.10"
port:    "22:22"
zip:     "02134"
```

CHECKING WHAT A PARSER SEES

```yaml
import yaml
yaml.safe_load("on: push")          # PyYAML = YAML 1.1 rules
yaml.safe_load("v: 3.10")           # {True: 'push'}   ← key is a bool!
yaml.safe_load('v: "3.10"')         # {'v': 3.1}
                                    # {'v': '3.10'}
```

YAML 1.1 VS 1.2 BY TOOL

TOOL / LIBRARY	RULES
PyYAML	1.1
ruamel.yaml	1.2 (default)
js-yaml 4 / yaml (npm)	1.2 core schema
Ruby Psych	1.1
SnakeYAML (Java)	1.1; SnakeYAML Engine = 1.2
go-yaml v3	1.2 mostly; `yes/no` → bool only into `bool` fields
Kubernetes (sigs.k8s.io/yaml)	1.1 booleans via go-yaml v2

CHARACTERS THAT FORCE QUOTING

STARTS WITH / CONTAINS	WHY
* &	alias / anchor
!	tag
| >	block scalar
{ [	flow mapping / sequence
@ backtick	reserved
%	directive (at line start)
– ?	sequence item / complex key
contains : or #	mapping value / comment

WHICH QUOTES?

STYLE	ESCAPES	USE FOR
plain	none	simple words
'single'	'' only	regex, Windows paths, literal text
"double"	\n \t \" \u…	special characters, explicit strings

COMMON MISTAKES

- Trusting a linter-clean file — `country: NO` is valid YAML; it is just a boolean in 1.1 parsers
- Writing tool versions unquoted (`node: 18.10`, `python-version: 3.10`) — they become floats and lose the trailing zero
- Leaving Compose port mappings unquoted — values under 60 like `22:22` are parsed as base-60 integers by 1.1 parsers
- Using `""` and empty values interchangeably — an empty value is `null`, not an empty string

TIPS

- Enable yamllint's `truthy` rule — it flags `yes`, `on`, `NO` and friends
- Round-trip a file through your real parser (`python -c "import yaml,sys; print(yaml.safe_load(sys.stdin))" < f.yml`) to see the types it gets
- Prefer YAML 1.2 libraries (ruamel.yaml, `yaml` npm) for new code
- When in doubt, quote — quoting a string never changes its meaning; not quoting sometimes does

SUMMARY

- `NO` / `yes` / `on` — booleans in YAML 1.1; quote country codes and words
- `3.10` — always a float; quote versions
- `22:22` / `010` — base-60 and octal in 1.1; quote ports and zip codes
- `~` / empty — null; `""` for empty string

Templating & Config Layering

YAML HAS NO VARIABLES

Plain YAML has no variables, includes, conditionals or loops — **anchors** are its only reuse feature. Every tool that needs more adds its own layer on top: **variable substitution** (Docker Compose), **text templating** that renders YAML from a template (Helm, Ansible/Jinja, GitHub Actions `${{ }}`), **file merging** (Compose override files, Helm values files) and **structural patching** (Kustomize). Knowing which layer you are in tells you what syntax works and when it runs.

WHICH APPROACH?

- **Substitution** — swap in environment values; no logic. Compose, many CI systems
- **Templating** — full logic (`if` , `range`) but the template is not valid YAML until rendered. Helm, Ansible
- **Merging** — later files override earlier ones key by key. Compose `-f` , Helm `-f`
- **Patching** — plain YAML base + overlays; nothing to render. Kustomize
- Inspect the final YAML: `docker compose config` , `helm template` , `kubectl kustomize`

NOTES

- → `${VAR:-default}` — default when unset **or empty**; `${VAR-default}` only when unset
- → `${VAR:?msg}` — stop with an error if missing; good for secrets
- → `$$` — escapes a literal dollar sign in Compose
- → `.env` — read automatically for substitution; not passed into containers unless listed
- → `compose.override.yaml` — merged on top of `compose.yaml` unless you pass explicit `-f` files
- → `{{- ... }}` — the dash trims whitespace before the tag; vital for correct indentation
- → `toYaml .Values.resources | nindent 12` — dump a values block as YAML, on a new line indented 12 spaces
- → `patches` — Kustomize merges each patch into the base resource with the same `kind` + `name`

DOCKER COMPOSE — SUBSTITUTION

```yaml
# compose.yaml   (values come from the shell or a .env file)
services:
  api:
    image: "myorg/api:${TAG:-latest}"     # default if unset or empty
    ports: ["${PORT:-8080}:8080"]
    environment:
      DB_URL: "${DB_URL:?DB_URL is required}"   # error if missing
      PRICE: "$$5"                            # literal $ → "$5"
```

HELM — TEMPLATES + VALUES

```yaml
# templates/deployment.yaml (Go template → YAML)
spec:
  replicas: {{ .Values.replicas | default 1 }}
  template:
    spec:
      containers:
        - name: api
          image: "{{ .Values.image.repo }}:{{ .Values.image.tag }}"
          resources: {{- toYaml .Values.resources | nindent 12 }}
```

KUSTOMIZE — BASE + OVERLAY

```yaml
# overlays/prod/kustomization.yaml
resources: [../../base]
namePrefix: prod-
images:
  - name: myorg/api
    newTag: "1.4.2"
patches:
  - path: replicas.yaml       # partial Deployment: spec.replicas: 5
```

COMPOSE SUBSTITUTION FORMS

FORM	RESULT
`${VAR}`	value of VAR (empty if unset)
`${VAR:-x}`	x if unset or empty
`${VAR-x}`	x only if unset
`${VAR:?msg}`	error if unset or empty
`${VAR:+x}`	x if VAR is set and non-empty

LAYERING ORDER (LATER WINS)

TOOL	ORDER
Compose	`compose.yaml` → override → each `-f` in order
Helm	chart `values.yaml` → each `-f` → `--set`
Kustomize	base resources → overlay patches
GitHub Actions	workflow `env` → job `env` → step `env`

COMMON MISTAKES

- Starting a value with `{{` unquoted (Ansible, Helm values) — YAML reads `{` as a flow mapping; write `"{{ var }}"`
- Using `${VAR}` in a Helm chart or `{{ }}` in Compose — each tool only understands its own syntax
- Forgetting `{{-` / `nindent` in Helm — rendered YAML ends up mis-indented and fails to parse
- Expecting arrays to merge across Compose or Helm files — lists are usually **replaced**, not appended

TIPS

- Run `docker compose config` / `helm template` in CI and diff the output — review the YAML that actually ships
- Keep secrets out of committed YAML: `${VAR:?}` in Compose, `--set` /external secrets in Helm
- Prefer Kustomize overlays when you only need small per-environment changes — the base stays plain YAML
- Use anchors (Sheet 05) for repetition inside one file before reaching for a template engine

SUMMARY

- `${VAR:-default}` — Compose substitution with fallback
- `compose.override.yaml` / `-f` — later files override earlier
- `{{ .Values.x }}` + `toYaml | nindent` — Helm templates
- `kustomization.yaml` — plain base + patch overlays

yq — Query & Edit YAML

yq eval · select() · = · |= · del() · -i · -o=json · ea · * merge · strenv()

JQ FOR YAML

yq (mikefarah/yq, v4) reads, queries and edits YAML from the command line with jq-like expressions. It keeps **comments and key order** when editing in place, handles multi-document files, and converts between YAML, JSON, XML, TOML, CSV and properties. It is the usual way to bump an image tag, read a value in a CI script, or merge environment-specific config files.

BEFORE YOU START

- Install: `brew install yq`, `snap install yq`, or the single binary from GitHub releases
- Check `yq --version` — a different Python package also called `yq` (kislyuk) wraps jq and uses jq syntax
- `yq 'expr' file` — print result; add `-i` to write back to the file
- Always single-quote the expression so the shell leaves `$`, `|` and `*` alone
- Output is YAML by default; `-o=json` for JSON, `-r` is on by default (no quotes around strings)

NOTES

- `.a.b` — path; `.[0]` index; `.[]` every item; `|` pipes the result to the next step
- `select(cond)` — keep only matching nodes, like `WHERE`
- `=` — set to a value; `|=` — update using the old value (`.` is the current value)
- `strenv(TAG)` — env var as a string; `env(TAG)` parses it as YAML (so `1.10` → float)
- `del(path)` — remove a key or array item
- `..` — recursive descent; `select(key == "image")` finds that key at any depth
- `ea` (eval-all) — load all files at once so they can be combined; default `e` processes files one by one
- `*` — deep merge maps; arrays from the right side **replace** the left (`*+` appends)

EVERYDAY COMMANDS

```
# ── Read ──────────────────────────────────────────────────────
yq '.spec.replicas' deploy.yaml                    # 3
yq '.services | keys' compose.yaml                 # - api   - db
yq '.spec.template.spec.containers[0].image' deploy.yaml
yq '.items[] | select(.kind == "Service") | .metadata.name' all.yaml
yq 'select(document_index == 1)' multi.yaml        # 2nd document only

# ── Edit in place (-i) ────────────────────────────────────────
yq -i '.spec.replicas = 5' deploy.yaml
yq -i '.image.tag = strenv(TAG)' values.yaml       # env var, kept as string
yq -i '.version |= . + 1' meta.yaml                # update relative to old value
yq -i '.tags += ["prod"]' meta.yaml                # append to array
yq -i 'del(.metadata.annotations)' deploy.yaml
yq -i '(.. | select(key == "image")) |= sub(":.*", ":1.4.2")' k8s.yaml

# ── Convert ───────────────────────────────────────────────────
yq -o=json '.' config.yaml > config.json           # YAML → JSON
yq -p=json -o=yaml '.' data.json                   # JSON → YAML

# ── Merge files (later wins) ──────────────────────────────────
yq ea '. as $f ireduce ({}; . * $f)' base.yaml prod.yaml
```

CORE OPERATORS

OPERATOR	DOES
`keys / length`	map keys, item count
`has("k")`	true if key exists
`sort_by(.name)`	sort array of maps
`unique / unique_by(.x)`	remove duplicates
`to_entries / from_entries`	map ↔ `[{key, value}]`
`sub("re", "new")`	regex replace in strings
`// "default"`	alternative if null/false
`... comments=""`	strip all comments

FLAGS

FLAG	MEANING
`-i`	edit file in place
`-o=json|yaml|xml|props|csv`	output format
`-p=json|xml|toml|csv`	input format
`-I 4`	indent width
`-e`	exit 1 if result is null/false (CI checks)
`-N`	no `---` between documents
`--front-matter=process`	edit YAML front matter in Markdown

MERGE VARIANTS

OPERATOR	ARRAYS	KEYS
`*`	replace	deep merge
`*+`	append	deep merge
`*d`	merge by index	deep merge
`*n`	replace	only add new keys
`*?`	replace	only update existing keys

COMMON MISTAKES

- Double-quoting the expression — the shell expands `$f`, `*` and backticks before yq sees them
- Using `env(VERSION)` for versions — `1.10` becomes the number `1.1`; use `strenv()`
- Running a jq-syntax command against mikefarah/yq (or the reverse) — check which `yq` is on the PATH
- Merging with `*` and expecting lists to combine — they are replaced unless you use `*+`

TIPS

- Use `yq -e '.spec.replicas > 1' deploy.yaml` as a CI assertion — non-zero exit when false
- Preview edits without `-i`, then rerun with `-i` once the output looks right
- `yq -P '.' file.json` pretty-prints any supported input as idiomatic YAML
- Pair with `kubectl get -o yaml | yq '...'` to slice live cluster objects

SUMMARY

- `yq '.a.b' f.yaml` — read a value
- `yq -i '.a = "x"' f.yaml` — edit in place, comments kept
- `select()` / `del()` / `|=` — filter, delete, update
- `-o=json` / `-p=json` — convert formats
- `yq ea '... * ...'` — merge files; later wins

Editor Schemas & CI Linting

CATCH MISTAKES BEFORE THEY RUN

Two layers of checking stop most broken YAML. In the **editor**, the Red Hat YAML extension (built on `yaml-language-server`) validates files against a **JSON Schema** — giving autocomplete, hover docs and red squiggles for wrong keys. Schemas come from **SchemaStore** automatically for common files, or from a comment or setting you add. In **CI**, `yamllint` checks syntax and style, and schema validators such as `check-jsonschema`, `actionlint` and `kubeconform` check meaning.

EDITOR SETUP (VS CODE)

- Install **YAML** by Red Hat — works for `.yml` and `.yaml`
- SchemaStore matching is on by default (`yaml.schemaStore.enable`): workflows, Compose, Dependabot, etc.
- A modeline comment in the file beats every other schema setting
- Kubernetes: map the built-in `kubernetes` schema to your manifest globs
- CloudFormation / custom tags: list them in `yaml.customTags` or every `!Ref` is an error

NOTES

- → `# yaml-language-server: $schema=` — per-file schema; also accepts a relative path
- → `"yaml.schemas"` — schema URL or path → glob(s) of files it applies to
- → `"kubernetes"` — special value for the extension's built-in Kubernetes schema
- → `!Ref scalar` — declares a custom tag and the node kind it wraps
- → `extends: default` — start from yamllint's default rules; `relaxed` is looser
- → `truthy` — flags `yes` , `on` , `NO` ; `check-keys: false` skips mapping keys

PICK A SCHEMA PER FILE

```yaml
# yaml-language-server: $schema=https://json.schemastore.org/github-workflow.json
name: CI
on: [push]
```

.VSCODE/SETTINGS.JSON

```json
{
  "yaml.schemas": {
    "kubernetes": "k8s/**/*.yaml",
    "./schemas/app-config.schema.json": "config/*.yaml",
    "https://json.schemastore.org/github-action.json": "action.yml"
  },
  "yaml.customTags": ["!Ref scalar", "!Sub scalar", "!GetAtt scalar", "!If sequence"]
}
```

.YAMLLINT

```yaml
extends: default
rules:
  line-length: { max: 120, level: warning }
  truthy:
    allowed-values: ["true", "false"]
    check-keys: false          # allow `on:` in GitHub workflows
  indentation: { spaces: 2, indent-sequences: consistent }
  document-start: disable
ignore: |
  charts/*/templates/          # Helm templates aren't plain YAML
```

.PRE-COMMIT-CONFIG.YAML

```yaml
repos:
  - repo: https://github.com/adrienverge/yamllint
    rev: v1.35.1
    hooks: [{ id: yamllint, args: [--strict] }]
  - repo: https://github.com/python-jsonschema/check-jsonschema
    rev: 0.29.4
    hooks:
      - id: check-github-workflows
      - id: check-jsonschema
        files: ^config/.*\.yaml$
        args: [--schemafile, schemas/app-config.schema.json]
```

VALIDATORS FOR SPECIFIC YAML

FILES	TOOL	COMMAND
any YAML (style/syntax)	yamllint	`yamllint -f github .`
any YAML + JSON Schema	check-jsonschema	`check-jsonschema --schemafile s.json f.yaml`
GitHub workflows	actionlint	`actionlint`
Kubernetes manifests	kubeconform	`kubeconform -strict -summary k8s/`
Compose files	docker compose	`docker compose config -q`
Helm charts	helm	`helm lint ./chart`
CloudFormation	cfn-lint	`cfn-lint template.yaml`

USEFUL YAMLLINT RULES

RULE	CATCHES
truthy	yes / on / NO booleans
key-duplicates	same key twice in one mapping
indentation	mixed or inconsistent indent
line-length	overlong lines
quoted-strings	enforce quote style (off by default)

COMMON MISTAKES

- Setting a `yaml.schemas` glob for a file that also has a modeline — the modeline wins, so the setting looks ignored
- Running yamllint on Helm templates — `{{ }}` isn't YAML; ignore `templates/` and lint `helm template` output instead
- Disabling `truthy` entirely to silence `on:` — use `check-keys: false` so `yes` / `no` values are still caught
- Copying pre-commit `rev:` values once and never updating — run `pre-commit autoupdate`

TIPS

- Write a JSON Schema for your own config files — the editor then autocompletes every key for the whole team
- `yamllint -f github` prints errors as inline annotations on pull requests
- Run `kubeconform` with `-strict` to reject unknown fields that `kubectl apply` would silently drop
- Commit `.vscode/settings.json` schema mappings so everyone gets the same validation

SUMMARY

- `# yaml-language-server: $schema=` — schema for one file
- `yaml.schemas` — schema → glob mapping in settings
- `.yamllint` — style and syntax rules for CI
- `check-jsonschema` / `kubeconform` / `actionlint` — schema checks in CI
- `pre-commit` — run all of them before every commit

Sheet 13

Quick Reference — Syntax & Types

structure · scalars · collections · quoting · block scalars · anchors & merges · tags · type gotchas

STRUCTURE

SYNTAX	DOES	#
`key: value`	mapping entry; space after `:`	01
`- item`	sequence entry	01
`# note`	comment to end of line	01
`---`	start of a document	01
`...`	end of a document	01
`2-space indent`	nesting; tabs not allowed	01

SCALARS

SYNTAX	DOES	#
`42 / 0x2A / 0o52`	integers	02
`3.14 / 1e3 / .inf`	floats	02
`true / false`	booleans (YAML 1.2)	02
`null / ~ / (empty)`	null	02
`2026-10-17`	timestamp (1.1 parsers)	02
`"42"`	force a string	02

COLLECTIONS

SYNTAX	DOES	#
`- a`	block sequence	03
`[a, b, c]`	flow sequence	03
`{x: 1, y: 2}`	flow mapping	03
`- name: x`	list of mappings	03
`? complex key`	explicit key	03
`[] / {}`	empty seq / map	03

QUOTING

SYNTAX	DOES	#
`plain text`	unquoted string	04
`'single'`	literal; `''` escapes a quote	04
`"double\n"`	escapes: `\n \t é`	04
`"key: value"`	quote when it has `:` or `#`	04
`"@x" / "*x"`	quote leading `@ * & ! %`	04
`"yes" / "on"`	keep 1.1 booleans as text	02

BLOCK SCALARS

SYNTAX	DOES	#
`|`	literal: keep newlines	04
`>`	folded: newlines → spaces	04
`|- / >-`	strip final newline	04
`|+ / >+`	keep all trailing newlines	04
`|2`	explicit indentation indicator	04

ANCHORS & MERGES

SYNTAX	DOES	#
`&name`	define an anchor	05
`*name`	alias: reuse the node	05
`<<: *base`	merge keys into a map	05
`<<: [*a, *b]`	merge several (first wins)	05
`x-common: &c`	Compose extension field	05

TAGS & DIRECTIVES

SYNTAX	DOES	#
`!!str 42`	force a core type	06
`!!int "7"`	cast to int	06
`!!binary`	base64 data	06
`!Ref / !Sub`	app-defined local tags	06
`%YAML 1.2`	version directive	06
`%TAG ! tag:x.com:`	tag shorthand	06

TYPE GOTCHAS

VALUE	1.1 PARSER READS	#
`NO / no / off`	false ("Norway problem")	02
`yes / on / y`	true	02
`012`	octal 10 in 1.1	02
`1:30`	sexagesimal 90 in 1.1	02
`3.10`	float 3.1, not "3.10"	02
`~ / null`	null, not the text	02

⚠ COMMON MISTAKES

- Tabs for indentation — YAML only allows spaces
- Unquoted `NO`, `on`, `3.10` or `012` changing type in YAML 1.1 parsers
- Missing space after `:` or `-`, so the line parses as one plain string

✓ TIPS & BEST PRACTICES

- Quote any value that only looks like a number, boolean or date
- Use `|` for scripts and `>-` for long one-line text
- Keep anchors in a top-level `x-` or `.base` key and merge with `<<:`

QUICK SUMMARY

- `key: value` maps, `- item` lists, indentation nests
- Quotes: single = literal, double = escapes
- `&anchor` / `*alias` / `<<:` — reuse
- `!!type` forces a type; `---` splits documents

Sheet 14

Quick Reference — Patterns & Tooling

Docker Compose · GitHub Actions · Kubernetes · yq · yamllint · parsers by language · schemas · YAML vs JSON

DOCKER COMPOSE

KEY	DOES	#
`services:`	containers to run	07
`image: / build:`	image name or build context	07
`ports: ["8080:80"]`	host:container (quote it)	07
`environment:`	env vars, map or list	07
`volumes:`	named volumes / bind mounts	07
`depends_on:`	start order	07

GITHUB ACTIONS

KEY	DOES	#
`on: [push]`	triggers	07
`jobs.<id>.runs-on`	runner image	07
`steps: - uses:`	reuse an action	07
`- run: |`	shell commands	07
`with: / env:`	inputs / variables	07
`${{ secrets.X }}`	expression, quote if leading	07

KUBERNETES

KEY	DOES	#
`apiVersion: / kind:`	resource type	07
`metadata.name`	object name	07
`metadata.labels`	selectable tags	07
`spec:`	desired state	07
`---`	several objects per file	07
`containers: - name:`	pod containers	07

YQ & YAMLLINT

COMMAND	DOES	#
`yq '.a.b' f.yml`	read a path	08
`yq -i '.a = 1' f.yml`	edit in place	08
`yq -o=json f.yml`	convert to JSON	08
`yq ea '. as $i ireduce…'`	merge files	08
`yamllint .`	lint style + syntax	08
`.yamllint`	rule config file	08

PARSERS

LANGUAGE	LOAD SAFELY	#
Python	`yaml.safe_load(f)`	08
JavaScript	`YAML.parse(s)` (yaml pkg)	08
Go	`yaml.Unmarshal(b, &v)`	08
Ruby	`YAML.safe_load(s)`	08
Java	SnakeYAML / Jackson YAML	08
C#	YamlDotNet `Deserialize`	08

YAML VS JSON

FEATURE	YAML	JSON
Comments	#	none
Superset	1.2 parses JSON	—
Anchors	& / ∗	none
Multiline	| and >	\n only
Quotes	mostly optional	required
Types	tags, 1.1 quirks	6 fixed

VALIDATION

TOOL	DOES	#
JSON Schema	validate structure	08
`# yaml-language-server`	`$schema=` modeline in editors	08
`check-jsonschema`	CLI schema check	08
`kubeconform`	Kubernetes manifests	08
`actionlint`	GitHub workflows	08
`docker compose config`	resolve + validate	08

COMMON ERRORS

MESSAGE	CAUSE	#
`mapping values not allowed`	`:` inside a plain value	04
`found character '\t'`	tab indentation	01
`did not find expected key`	bad indent level	01
`found undefined alias`	∗x before &x	05
`duplicate key`	same key twice in a map	03

⚠ COMMON MISTAKES

- `yaml.load` without a safe loader — can build arbitrary objects
- Unquoted port mappings like `22:22` (sexagesimal in YAML 1.1)
- Assuming merge keys `<<:` work everywhere — they are not in YAML 1.2 core

✓ TIPS & BEST PRACTICES

- Add a `$schema` modeline so editors autocomplete and validate
- Run `yamllint` and the tool's own validator in CI
- Convert with `yq -o=json` to see exactly what a parser reads

QUICK SUMMARY

- Compose, Actions and Kubernetes are plain YAML maps
- `yq` reads and edits; `yamllint` checks style
- Always use the safe loader in code
- Validate with JSON Schema or the tool's checker